

Centro Educativo Técnico Laboral Kinal

Área de Diversificado

Taller

Asesor: Elmer Santos



Documentación Sistema Bancario

Presentado Por:

Jefry André Cruz Yumán – Grupo #6

6to Perito en Informática

IN6BM

Matutina

27/02/2026

Documentación referente a nuestro Sistema Bancario

Introducción

En este proyecto buscamos en poder desarrollar un sistema bancario el cual consiste en solo backend por el momento, esta estructurado bajo una arquitectura modular orientada a microservicios, los cuales simulan el operativo de una institución financiera que se aloja de forma digital.

Objetivos del proyecto

El sistema bancario tiene como objetivo principal moldear procesos bancarios reales aplicando lo siguiente:

- Control de acceso basado en roles
- Seguridad mediante JWT
- Manejo estructurado de transacciones
- Registro de operaciones

También el proyecto busca aplicar buenas prácticas de desarrollo de backend.

Alcance del sistema

Este proyecto como tal nos permite:

- Registro de usuarios con su autenticación por medio de Tokens
- Gestión de roles (ADMIN, MANAGER, USER, ATM)
- Creación y administración de cuentas bancarias
- Depósitos y retiros con validaciones e implementación de divisas
- Transferencias entre cuentas con divisas
- Gestión de préstamo
- Administración de tarjetas
- Bloqueo de cuentas
- Creación de nuevas monedas aparte de las que están en el banco.
- Creación de estados de cuenta

Por el momento este proyecto no tiene la parte grafica o sea el frontend, porque esta diseñado como API RESTful para ser consumido por medio de endpoints en postman.

Arquitectura General

En si el proyecto lleva una arquitectura basada en:

- Patron de diseño MVC (Modelo, Controller, Routes)
- Separación por dominios funcionales
- Middlewares para validación y seguridad
- Helpers para lógica reutilizable y tener un controlador más limpio

Una parte importante es que el proyecto esta dividido en dos servicios y esto permite:

- Separa autenticación de lógica financiera
- Mayor control de seguridad
- Posibilidad de escalar servicios individualmente

Enfoque técnico

El sistema este compuesto por varias tecnologías que conviven mutuamente como:

- Node.js
- MongoDB
- Mongoose
- JSON Web Token (JWT)
- Arquitectura modular
- Validaciones de roles por Middlewares

El diseño prioriza:

- Seguridad
- Escalabilidad
- Mantenibilidad del código
- Integridad de datos
- Reglas financieras

Alcance del sistema explicación

- **Registro de usuarios con su autenticación por medio de Tokens**
- **Gestión de roles (ADMIN, MANAGER, USER, ATM)**

Nuestro proyecto del sistema bancario cuenta con un servicio de autenticación llamado AuthService el cual esta creado a partir de:

- Node.js
- Express.js
- PostgreSQL
- JWT
- Cloudinary
- Servicios de Email
- Arquitectura MVC
- Middlewares
- Docker

Por lo tanto, es un sistema de autenticación robusto a continuación veremos el uso de este servicio por medio de endpoints y tokens.

Primero tenemos que ingresar los datos de nuestro usuario para registrarnos como nombre, apellido, nombre de usuario, correo, contraseña, teléfono y foto de perfil(opcional), al ingresar estos datos en from-data y tener nuestro endpoint post <http://localhost:3005/api/v1/auth/register> nos creara nuestro registro y nos mandaran un token al email que registramos.

Luego de registrarnos nos toca verificar nuestro token que nos llegó por correo, este token lo tenemos que ingresar en nuestro get-token-login con la ruta <http://localhost:3005/api/v1/auth/verify-email>, que nos verificara nuestro email para poder hacer nuestro login.

Al tener confirmado nuestro correo podemos irnos al apartado de login en el cual nos deja entrar a nuestro sistema de autenticación con la ruta <http://localhost:3005/api/v1/auth/login> y en este tenemos que ingresar solamente nuestro email y password para poder entrar y tener un login exitoso y de que nos sirve esto mas que todo para saber que role tenemos, nuestro userId y nuestro token para poder trabajar dentro del sistema bancario.

Algunos detalles mas que tiene nuestro AuthService es poder cambiar el role de un usuario, volver a mandar la verificación de nuestro email en dado caso que no nos llegue el token la primera vez que nos registramos, un forgot Password si en dado caso perdemos nuestra contraseña este manda un token a nuestro correo y ese token lo tenemos que ingresar dentro de nuestro New Password este nos pide nuestro token junto con nuestra nueva contraseña y por ultimo tenemos el get profile que solo lista tu usuario.

- **Creación y administración de cuentas bancarias**

Referente a la creación de cuentas y a su administración, el sistema bancario cuenta con su crud completo para realizar estas gestiones en el primero tenemos nuestra creación de cuenta las cuentas bancarias que únicamente se pueden crear por un administrador en estas cuentas ingresamos el tipo de cuenta, monto de la cuenta, día de apertura, limite de retiros, tasa de intereses anuales, tipo de moneda, dpi, dirección, trabajo, sueldo, teléfono, userId, estado de cuenta (activa, inactiva) esto lo ingresamos por medio de nuestro raw JSON con la ruta post <http://localhost:3006/api/v1/accounts/create> esto nos permite crear una cuenta nueva para los diferentes usuarios.

Para nuestro método de listar este lista únicamente si cuentas con el role de admin o manager ya que estos son los que tienen el control sobre las cuentas del banco y se lista por medio de un get con la ruta <http://localhost:3006/api/v1/accounts/>.

Para nuestro método de actualizar este si permite que todos podamos actualizar nuestra cuenta, pero para el caso del usuario únicamente puede actualizar su nombre, dirección, nombre del trabajo y el pago mensual pero igual que en el crear cuenta el pago mensual tiene que ser igual a 100 o mayor y este lo podemos usar por medio de un put con la ruta <http://localhost:3006/api/v1/accounts/ACC-000-000>

Para nuestro buscar por número de cuenta solo tenemos que ingresar nuestro numero de cuenta en la ruta y dejar que muestre nuestros datos de cuenta por medio de un get con la ruta <http://localhost:3006/api/v1/accounts/ACC-000-0000>.

Y casi terminando este cuenta también con un eliminar cuenta el cual solo lo puede hacer un admin, manager o cajero con la ruta <http://localhost:3006/api/v1/accounts/ACC-000-0000>.

Ya por último este cuenta con un apartado para actualizar el estado de la cuenta por medio de un patch <http://localhost:3006/api/v1/accounts/ACC-000-0000/status>.

- **Depósitos y retiros con validaciones e implementación de divisas**

En este apartado comenzaremos con depósitos el cual consta de un post con la ruta <http://localhost:3006/api/v1/deposits/create> y en el raw JSON solo tenemos que agregar el numero de la cuenta a la que queremos hacerle el deposito junto con el monto, la descripción y el tipo de moneda que queremos que sea el deposito ya sea en euros, dólares, quetzales, lempiras, soles, yenes, etc.

En el apartado de los retiros este cuenta con un post con la ruta <http://localhost:3006/api/v1/withdrawal/> y en nuestro raw JSON solo tenemos que agregar el numero de la cuenta de la que queremos sacar dinero junto con el monto a sacar, pero retiros cuenta con la función de que si no es mi cuenta me lanza error diciendo que esta cuenta no existe o no me pertenece.

- **Transferencias entre cuentas con divisas**

En este otro apartado de transferencias o transacciones esta entidad cuenta con su crud completo, empecemos con el crear transacción este contiene su ruta <http://localhost:3006/api/v1/transaction/create> en la cual en su raw JSON tenemos que agregar sus atributos como: cuenta que manda la transacción, cuenta que recibe, tipo de transacción, monto, tipo de moneda de la cuenta que manda la transacción, descripción, estado, favorito, alias, userId luego de llenar los campos se podrá ver la transacción o transferencia creada exitosamente.

Como otra parte tenemos el listar que este cuenta con su ruta <http://localhost:3006/api/v1/transaction/> y este únicamente puede ser utilizado si tu role es admin.

También contamos con nuestro buscar y actualizar que los dos funcionan con la misma ruta <http://localhost:3006/api/v1/transaction/:id> y en el actualizar podemos modificar todo lo de nuestra transacción.

También tenemos nuestro eliminar transacción que tiene su ruta <http://localhost:3006/api/v1/transaction/:id>.

Y por último tenemos nuestro apartado de favoritos y este funciona por medio de nuestro favorito en el crear si a nuestra transacción dejamos favorito:true entonces el alias de esa transacción junto con el numero de cuenta, nombre del usuario se guardarán y se listarán en el list fav con esta ruta get <http://localhost:3006/api/v1/transaction/favorites>.

- **Gestión de préstamo**

Referente a la gestión de un préstamo esta cuenta también con su crud completo empezando con el crear que este nos pide la información de la cuenta también nos pide el monto del préstamo junto con la cantidad de meses disponibles para pagar, tasa de interés entre otros, este método tiene su ruta que es <http://localhost:3006/api/v1/loan/create>.

Pasando al siguiente tenemos nuestro listar préstamo que este se basa en listar todos los prestamos existentes con esta ruta get <http://localhost:3006/api/v1/loan> y estos solo se listan si tienes el role admin, manager o atm/cajero.

También contamos con nuestro eliminar, buscar y actualizar que también trabajan con la misma ruta <http://localhost:3006/api/v1/accountLocks/:id> pero diferente tipo de método este también controla los roles de usuario.

- **Administración de tarjetas**

Esta administración de tarjetas cuenta también con su crud completo empezando por el create que solamente se pueden crear con el role admin y manager, estas tarjetas nos piden ingresar userId, tipo de tarjeta, cvv, monto de la tarjeta, fecha de expiración y el pin en el raw JSON con la ruta <http://localhost:3006/api/v1/cards/create>.

Para nuestro método de listar esta cuenta con la validación de roles donde si yo soy un user_role no me deje listar nada porque solo lo pueden hacer los roles admin con la ruta get <http://localhost:3006/api/v1/cards/>.

Referente a nuestro actualizar este solamente se puede realizar si tenemos el role de admin o de manager lo que da a entender que si queremos cambiar nuestro pin de la tarjeta tenemos que comunicarnos con servicio al cliente para que nos modifiquen nuestro pin y se trabaja con un put y con la ruta <http://localhost:3006/api/v1/cards/:id>.

también contamos con el buscar tarjeta esta cuenta con la validación que verifica si la tarjeta nos pertenece para poder buscarla y listarla su ruta es <http://localhost:3006/api/v1/cards/:id> por cierto esta misma ruta se usa para el eliminar solo que en el eliminar si se necesita role admin o manager.

- **Creación de nuevas monedas aparte de las que están en el banco.**

En este apartado como todos los demás no se pierde nada porque igual tiene el crud completo y este es por si en dado caso el banco quiere tener una moneda más de las que ya tiene por default y también solo el role admin puede agregar mas monedas al sistema bancario sus rutas son:

- Post: <http://localhost:3006/api/v1/coins/create>
- Get: <http://localhost:3006/api/v1/coins/>
- Put: <http://localhost:3006/api/v1/coins/:id>
- Get: <http://localhost:3006/api/v1/coins/:id>
- Delete: <http://localhost:3006/api/v1/coins/:id>

- **Creación de estados de cuenta**

Ya para ir terminando la documentación este proyecto cuenta con la creación de estados de cuenta en que consiste este consiste en un buscar por numero de cuenta y que hace, crea un documento pdf con los datos de la cuenta seleccionad junto con sus últimos movimientos financieros, también cuenta con un par de verificaciones tales como que no me deja crear estados de cuentas de otras personas y este pdf solicitado llega por correo el cual ya se registro previamente en la cuenta bancaria solo quedaría imprimir o descargar el documento y ver nuestro estado de cuenta con la ruta <http://localhost:3006/api/v1/accountStatements/account/ACC-000-0000/pdf>.

Para terminar este proyecto demuestra la aplicación de buenas prácticas como la separación de proyectos para tener una mejor responsabilidad, validaciones concretas, autenticación mediante JWT, control de roles y organización estructural por medio del patrón MVC.

Además considero que se logro simular el proceso de un sistema bancario obviamente un sistema bancario real lleva mucho más, pero esto es lo mas parecido que pudimos replicar utilizando nuestra propia lógica de negocio y programación.

Este proyecto por el momento no ha terminado aun falta la parte del frontend lo cual lo llegaremos a abordar mas adelante en el mismo proyecto.